

Encoder–IMU Fusion for Stance-Leg Odometry

A Worked Derivation

Arda Gencer

1 What we actually have, and why neither source is enough alone

At every control tick we have two independent raw readings: the three joint encoder angles per leg, $(\theta_{\text{abad}}, \theta_{\text{hip}}, \theta_{\text{knee}})$, and the IMU’s orientation, from which we extract pitch θ and roll φ . Neither one by itself tells us what we actually want, which is the body’s velocity and height in the world frame.

The encoders alone are not enough: forward kinematics on the joint angles tells us exactly where a foot sits relative to the body’s own reference frame, but it says nothing about which way that frame is tilted relative to the ground. A leg held at a fixed configuration produces the exact same joint angles whether the robot is standing level or pitched forward by 30° – the encoders cannot see the difference.

The IMU alone is not enough either: pitch and roll tell us how the body frame is tilted, but on their own they say nothing about where the feet are or whether the robot is translating at all. Orientation and position are simply different quantities.

So the two sources are complementary in a very literal sense – one gives a vector, the other gives the rotation needed to place that vector correctly in the world. Fusing them here means exactly that: computing a vector from the encoders, then rotating it with the IMU’s angles. Nothing probabilistic is happening; this is a deterministic geometric composition, not a Kalman-style statistical fusion.

2 Step 1 – encoders give a body-frame vector

Forward kinematics (the algebraic inverse of the leg’s inverse-kinematics solver) converts one leg’s joint angles into the foot’s position relative to that leg’s ABAD pivot, in the body’s own axes:

$$(f_x, f_y, f_z) = \text{FK}_{3D}(\theta_{\text{abad}}, \theta_{\text{hip}}, \theta_{\text{knee}}; o_y, s, L_1, L_2, D_{\text{ABAD}}) \quad (1)$$

where L_1, L_2 are the leg’s link lengths, D_{ABAD} is the fixed ABAD-to-hip arm length, and o_y, s encode which side (left/right) and end (front/back) of the robot the leg is mounted on. Adding that leg’s fixed mounting offset (x_0, y_0) from the body origin gives the foot position relative to the body origin itself,

$$\mathbf{p}_{\text{foot}}^B = (f_x + x_0, f_y + y_0, f_z). \quad (2)$$

The superscript B is deliberate: this vector is only correct when read in body-frame axes. We do not yet know how those axes sit relative to the world.

3 Step 2 – the IMU supplies the missing rotation

All three orientation angles – pitch θ , roll φ , and yaw ψ – come straight from the IMU’s orientation quaternion. An earlier version of this estimator dropped yaw entirely (assumed $\psi = 0$, pinning

the world x - y axes to wherever the robot happened to be facing when the estimator started); Section 4 discusses why that was a real bug and what using the full angle does and does not fix. The body-to-world rotation used is

$$R(\psi, \theta, \varphi) = R_z(\psi) R_y(\theta) R_x(\varphi) = \begin{pmatrix} \cos \psi \cos \theta & \sin \varphi \sin \theta \cos \psi - \sin \psi \cos \varphi & \sin \varphi \sin \psi + \sin \theta \cos \varphi \cos \psi \\ \sin \psi \cos \theta & \sin \varphi \sin \psi \sin \theta + \cos \varphi \cos \psi & \sin \psi \sin \theta \cos \varphi - \sin \varphi \cos \psi \\ -\sin \theta & \sin \varphi \cos \theta & \cos \varphi \cos \theta \end{pmatrix}, \quad (3)$$

with θ = pitch, φ = roll, ψ = yaw (composed as roll, then pitch, then yaw – matrix composition applies right to left, so a body-frame vector is rotated by roll first, then pitch, then yaw). Applying it to the encoder-derived vector re-expresses the same physical offset in world-frame axes:

$$\mathbf{p}_{\text{foot}}^W = R(\psi, \theta, \varphi) \mathbf{p}_{\text{foot}}^B = (w_x, w_y, w_z). \quad (4)$$

This single matrix multiplication is the entire fusion step: an encoder-derived vector, rotated by an IMU-derived matrix.

4 What the yaw fix does, and does not, fix

An earlier version of R used only $R_y(\theta)R_x(\varphi)$, silently assuming $\psi = 0$ at every tick. That is not a small simplification: it means the estimator’s own notion of “world x ” is really “whichever direction the robot happened to face when the estimator started,” frozen in place even as the robot actually turns. Since the height read-out is yaw-invariant (Section 5), the bug was invisible there – but it silently rotated every horizontal velocity and position estimate by however much the robot had actually yawed, growing worse the longer a run went on or the more the robot turned. The worked example in Section 7 shows this concretely: at roughly 20° of accumulated yaw, the estimator’s reported speed is barely affected, but its reported *direction* is off by almost exactly that same 20° – a heading-dependent, growing error, not a rounding artifact. Using the full $R(\psi, \theta, \varphi)$ instead removes this source of error entirely, for whatever value of ψ is supplied.

That last clause matters. Fixing the rotation *math* is not the same as fixing where ψ comes from. In this implementation, ψ is read from the same IMU orientation quaternion as θ and φ – which, as established elsewhere in this project, is the simulator’s exact ground-truth attitude, not a value derived from any sensor-fusion process. On real hardware this would not be available for free: Bloesch et al. (*State Estimation for Legged Robots – Consistent Fusion of Leg Kinematics and IMU*, RSS 2012) show that yaw and absolute position together form a manifold that is *provably unobservable* from IMU and leg kinematics alone – only roll, pitch, and body velocity can be recovered from that sensor set. Recovering yaw for real would require an additional absolute reference the robot does not have in this project, such as a magnetometer (workable in principle, but vulnerable in practice to interference from the leg actuators’ own motor currents) or a vision/LIDAR-based heading estimate. So this fix should be read precisely: it corrects a genuine bug in how a known orientation is used, while leaving unchanged – and now more clearly documented – the fact that the yaw value itself remains a simulation privilege rather than something this sensor suite could produce on a real robot.

5 Height falls out of a flat-ground constraint

If the ground is flat at world $z = 0$ and the current foot is planted on it, then the body’s height plus the foot’s world-frame z -offset from the body must sum to zero:

$$z_{\text{body}} + w_z = 0 \implies z_{\text{body}} = -w_z. \quad (5)$$

Notice this is a single algebraic read-out of Step 2’s result – nothing is integrated, so this estimate carries no accumulating drift; each tick is a fresh, independent computation. It is also, conveniently, completely insensitive to yaw: a rotation about the vertical axis only ever mixes the x and y components of a vector into each other, it can never move anything into or out of z . So whether or not yaw is tracked correctly, height comes out identically – the yaw bug discussed in Section 4 affects horizontal velocity and position only.

6 Velocity comes from differentiating a constancy constraint

The physical statement that gives us velocity is: while a foot is planted and not slipping, its world position does not move. Writing the body’s world position as $\mathbf{p}_{\text{body}}^W(t)$,

$$\mathbf{p}_{\text{body}}^W(t) + R(t) \mathbf{p}_{\text{foot}}^B(t) = \text{constant}, \quad (6)$$

where $R(t)$ is shorthand for the full rotation of Eq. (3), evaluated at whatever (ψ, θ, φ) the IMU reports at time t . Differentiating both sides with respect to time,

$$\mathbf{v}_{\text{body}}^W(t) = -\frac{d}{dt} [R(t) \mathbf{p}_{\text{foot}}^B(t)] = -[\dot{R}(t) \mathbf{p}_{\text{foot}}^B(t) + R(t) \dot{\mathbf{p}}_{\text{foot}}^B(t)], \quad (7)$$

by the product rule. There are genuinely two contributions here: one from the leg’s own joint angles changing ($\dot{\mathbf{p}}_{\text{foot}}^B$), and one from the *body itself rotating* while the foot vector is held ($\dot{R}$). The implementation keeps only the first term, approximating the body-frame derivative with a finite difference and applying the *current* tick’s rotation to it:

$$\mathbf{v}_{\text{body}}^W \approx -R_k \cdot \frac{\mathbf{p}_{\text{foot}}^B(t_k) - \mathbf{p}_{\text{foot}}^B(t_{k-1})}{\Delta t}. \quad (8)$$

To see exactly what this drops, compare it against differencing the two already-rotated positions instead, $[R_k \mathbf{p}_k - R_{k-1} \mathbf{p}_{k-1}]/\Delta t$. Expanding that:

$$\frac{R_k \mathbf{p}_k - R_{k-1} \mathbf{p}_{k-1}}{\Delta t} = R_k \frac{\mathbf{p}_k - \mathbf{p}_{k-1}}{\Delta t} + \frac{(R_k - R_{k-1}) \mathbf{p}_{k-1}}{\Delta t}, \quad (9)$$

and the second term is exactly a finite-difference approximation of the $\dot{R} \mathbf{p}_{\text{foot}}^B$ term missing from Eq. (8). So the estimator is implicitly assuming the body’s orientation does not change appreciably within one control tick ($\Delta t = 0.02$ s) – a small-rotation-per-timestep approximation. It is reasonable for a walking gait at 50 Hz, but it is a genuine, identifiable source of extra error during fast pitch/roll motion (for instance, right at footfall), not merely floating-point noise.

Once $\mathbf{v}_{\text{rel}}^W = R_k(\mathbf{p}_k - \mathbf{p}_{k-1})/\Delta t$ is computed, the stance-foot constraint flips its sign: the foot *appearing* to move by $\mathbf{v}_{\text{rel}}^W$ relative to the body means the body itself is moving by exactly the opposite,

$$\mathbf{v}_{\text{body}}^W = -\mathbf{v}_{\text{rel}}^W. \quad (10)$$

7 Worked example

Take the front-left leg ($o_y = +1$, $s = +1$), with $L_1 = L_2 = 0.2$ m, $D_{\text{ABAD}} = 0.1$ m, and mounting offset $(x_0, y_0) = (0.15, 0.113)$ m. Suppose two consecutive ticks, 20 ms apart, read:

	θ_{abad}	θ_{hip}	θ_{knee}		pitch θ	roll φ	yaw ψ
tick $k-1$	0.050	-0.100	0.300	tick $k-1$	0.040	0.015	0.350
tick k	0.050	-0.080	0.290	tick k	0.050	0.020	0.360

The robot has turned about 20° since the estimator started ($\psi \approx 0.35\text{--}0.36\text{ rad}$) – exactly the situation Section 4 shows breaks the yaw-dropped version of this estimator.

Step 1. Forward kinematics at each tick gives the foot position relative to the body origin, in body-frame axes – this step never involves the IMU, so it is unchanged from before:

$$\mathbf{p}_{\text{foot}}^B(t_{k-1}) = (0.169767, 0.132618, -0.389523) \text{ m}, \quad (11)$$

$$\mathbf{p}_{\text{foot}}^B(t_k) = (0.175709, 0.132615, -0.389475) \text{ m}. \quad (12)$$

Since the ABAD angle barely changed between ticks (both 0.05 rad) while hip and knee opened up slightly, almost all of the change shows up in f_x – exactly what we’d expect: swinging the hip/knee in their own plane moves the foot mostly along the leg’s forward direction, not sideways.

Step 2. Rotating each by that tick’s own full $R(\psi, \theta, \varphi)$ (Eq. (3)):

$$\mathbf{p}_{\text{foot}}^W(t_{k-1}) = (0.097319, 0.182905, -0.393968) \text{ m} \quad \Rightarrow \quad z_{\text{body}} = 0.393968 \text{ m}, \quad (13)$$

$$\mathbf{p}_{\text{foot}}^W(t_k) = (0.096698, 0.186390, -0.395043) \text{ m} \quad \Rightarrow \quad z_{\text{body}} = 0.395043 \text{ m}. \quad (14)$$

Height comes out *identical* to the yaw-free version of this example from an earlier draft – 0.393968 m and 0.395043 m again, to six decimal places – exactly confirming Section 4’s claim that yaw cannot affect z . The horizontal components, w_x and w_y , are completely different from that earlier draft, though, since a 20° heading change substantially redistributes a vector between x and y .

Step 3 – velocity. The body-frame finite difference is unchanged from Step 1 (it doesn’t involve the IMU either):

$$\frac{\mathbf{p}_{\text{foot}}^B(t_k) - \mathbf{p}_{\text{foot}}^B(t_{k-1})}{\Delta t} = (0.297093, -0.000119, 0.002378) \text{ m/s}. \quad (15)$$

Rotating this by the current tick’s full $R(\psi_k, \theta_k, \varphi_k)$ gives $\mathbf{v}_{\text{rel}}^W = (0.277871, 0.104413, -0.012476) \text{ m/s}$, so the estimated body velocity is the sign-flipped result,

$$\mathbf{v}_{\text{body}}^W = (-0.277871, -0.104413, 0.012476) \text{ m/s}. \quad (16)$$

For comparison, feeding the exact same inputs through the *old*, yaw-dropped rotation (i.e. Eq. (3) with ψ forced to 0) gives $\mathbf{v}_{\text{body}}^W = (-0.296840, 0.000167, 0.012476) \text{ m/s}$ instead – nearly the same *speed* ($\sqrt{v_x^2 + v_y^2} \approx 0.297 \text{ m/s}$ either way) but pointed in a direction that is wrong by about 20.6° , matching the robot’s actual yaw almost exactly. That is the bug in a single number: dropping yaw does not corrupt how fast the estimator thinks the body is moving, only which direction it thinks that motion is in – which is exactly what you’d expect from a rotation matrix that’s missing one of its three angles.

8 Combining legs, and the final integration

When more than one foot is in stance simultaneously, each produces its own independent ($z_{\text{body}}, \mathbf{v}_{\text{body}}^W$) estimate via the steps above. These are combined with a plain, unweighted arithmetic mean. It is worth being honest that this is a simplification: a leg near full extension is geometrically more

sensitive to a given joint-angle error than one near mid-range, so a statistically rigorous fusion would weight each leg’s contribution by its estimated reliability (e.g. a weighted least-squares or Kalman update) rather than trusting every stance leg equally.

Finally, horizontal position is obtained with a single Euler integration of the fused velocity,

$$x_{\text{body}}(t_k) = x_{\text{body}}(t_{k-1}) + v_x \Delta t, \quad (17)$$

and likewise for y . This is the only integration anywhere in the pipeline – height never accumulates error since it is read fresh each tick, and velocity itself is obtained by differentiation rather than integration, so horizontal position is one integration away from a non-drifting quantity rather than two integrations away from a double-drifting one, as a raw accelerometer-based estimate would be.